

DRAIN SINKHORN: 批量熵正则最优传输的安全排出*

Xinyang Wen

摘要

快速OT 后端降低了单次Sinkhorn 更新的成本, 但固定宽度批量执行仍会保持完整宽度运行, 直到其中最慢的问题完成。我们提出DRAIN SINKHORN: 一个面向批量熵正则最优传输的执行层, 它动态移除已经完成的问题, 并使后续更新变窄。该方法以独立Sinkhorn 问题的标准批量执行为起点。一次交替缩放循环后, 其中一侧边缘由构造保证为最新; 对另一侧边缘进行批量检查, 可筛选出需要执行双边验证器的问题。通过这一已有释放规则的问题会从所有逐问题张量中移除, 未完成的问题则继续执行。

分析将计算机会与硬件转化分开。对于首次通过的迭代编号 $n_1, \dots, n_W$ 和存活宽度 $A_k = |\{t : n_t \geq k\}|$, 静态执行消耗 $W n_{\max}$ 个问题更新槽位, 而动态压缩恰好消耗 $\sum_k A_k = \sum_t n_t$ 。若 $c_k(a, \xi_k)$ 表示宽度为 a 时更新的实测成本, 则匹配路径的精确时间差等于宽度成本节省 $\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)]$ 减去增量控制开销。因此, 深度异质性创造可移除工作, 而硬件宽度成本曲线决定其中多少能够转化为墙钟加速。在实测A100/Triton 配置上, 宽度扫描呈现波次形状的成本曲线: 当 $n = 1024$ 时, 宽度一到六处于一个近似平坦的区域, 而更大的问题会更早地响应宽度缩减。

匹配的固定宽度/动态压缩对照在四worker MetroPT-3 端点上得到1.746 $\times$, 在ImageNet-32 训练solver 字段上得到1.054 $\times$, 并在已登记的MetroPT scale-out 单元中最高达到3.110 $\times$ 。OTT-JAX 实现在三个容差上得到1.366–1.581 $\times$ 。当多个问题使用同一初始化时, 匹配的packed Triton 动态压缩达到1.421 $\times$, 并移除30.62% 的问题更新槽位。Packer19 提供匹配的逻辑mask 对照, 以及压缩收益在深度同步时消失的边界。实际残差和已登记应用端点共同构成实现证据; 所有报告的消费者质量门和训练质量门均通过。

1 引言

熵正则最优传输 (EOT) 的GPU 实现在执行单次Sinkhorn 更新方面已经显著进步。无矩阵算子、分块reduction 和融合流式kernel 降低了显存流量, 并避免实例化transport matrix [4, 5]。然而, 许多workload 会为预测性维护、科学状态转移或minibatch 训练准备一批coupling。当内部更新已经很快时, 总成本还取决于每个问题在批次中停留多久。

不同问题通常在不同迭代到达停止容差。静态批次会一直运行到需要最多迭代的问题完成: 已经完成的问题仍继续占据状态、验证检查带宽和kernel lane。逻辑mask 会冻结其状态, 但不会缩小融合kernel 看到的

张量。因此, 每个问题通过检查时的迭代编号与其中最大编号之间未被使用的矩形区域, 构成了一种独立的OT 工作量。

我们提出DRAIN SINKHORN, 一个移除这种padding 的执行层。该方法以独立Sinkhorn 问题的标准批量执行为起点。问题通过所选求解器的停止检查后, 系统将其从后续计算中移除。一次完整交替循环在理想算术下使刚更新的边缘保持为最新, 因此另一侧边缘可以提供廉价的批量预检。随后通过双边验证器的问题会从势函数、边缘、支撑描述符和scratch buffer 中移除。下一次后端更新以未完成集合的宽度运行。

这一设计将内部kernel 加速与跨问题执行分离。内部后端决定一次稳定化更新的成本; DRAIN SINKHORN 则改变一批问题实际执行的更新数量和宽度。我们使用packed Triton kernel、OTT-JAX 和PyKeOps 实现这一执行层。Packed Triton 路径还使用当前边缘进行初步检查; OTT-JAX 与PyKeOps 路径使用各自独立的后端机制移除已完成问题。

对问题 t , 令 n_t 为它首次在已配置验证时点通过双边验证器时的批量迭代编号。固定宽度执行消耗 $W n_{\max}$ 个问题更新槽位, 而动态压缩执行恰好消耗 $\sum_t n_t$ 。这个任务无关的恒等式只依赖观测到的迭代编号。随机形式进一步明确条件: 当且仅当某个窗口在某次批量迭代前以正概率同时包含已完成与未完成问题时, 期望可移除工作量为正。可移除槽位只是计算机会。若 $c_k(a, \xi_k)$ 表示更新 k 在宽度 a 时的实测成本, 则墙钟节省等于 $\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)]$ 减去增量控制开销。因此, 存活曲线与后端宽度成本曲线共同决定实际加速。

GPU 宽度成本会受到调度波次的量化影响, 而不一定与问题数成比例。Packed Triton 更新会启动数量随宽度变化的thread block; 当存活网格降至更低的成本层级时, 缩小 A_k 才会节省时间。求解器的验证器给出存活轨迹, 实测宽度扫描给出硬件转化曲线。

本文从常规固定宽度批量执行开始评估贡献: DrainSinkhorn 在保持所选内部后端不变的条件下, 增加由停止规则触发的释放与动态压缩。OTT-JAX 和PyKeOps 提供独立后端实现。实验区分OT 阶段加速和应用端点: 前者测量EOT 内部被移除的工作, 后者检验这一加速能否在消费者或模型工作保持不变时传递到完整应用。

本文贡献如下:

1. **批量Sinkhorn 执行中的动态压缩。** 本文以标准固定宽度batching 和所选求解器的停止规则为起点, 把已完成问题从全部逐问题状态中动态移除。之后的每个kernel 都按未完成集合的宽度执行。
2. **机会与转化核算。** 生存曲线恒等式计算被移除的问题更新槽位。精确的匹配路径时间分解把存活宽度映射到后端宽度成本曲线, 并减去增量控制开销, 从而分

*代码: github.com/cis-aconiticacid/DrainSinkhorn

离问题侧机会与硬件转化。

3. **组件因果证据**。匹配对照分离批量检查、Sinkhorn 专用screening、逻辑masking 和动态压缩。在保持总迭代次数不变的条件下进行分组实验，并结合直接宽度成本测量，识别动态压缩获利所需的两个条件。
4. **后端与端点证据**。MetroPT-3、Packer19 和ImageNet-32 评估packed Triton 路径；OTT-JAX 和PyKeOps 在ImageNet-32 与A2D2 上测试迁移。这些路径在所测试workload 上优于使用相同后端的指定参考路径。数值行为通过parity、残差和有限精度压力测试报告，完整消费者则检验应用可见输出和训练质量。

快速kernel 让一次更新更便宜。动态压缩移除原本只会继续发给已完成问题的更新。

2 问题设定与已有系统

嫡正则最优传输。对于正的单位质量边缘分布 $a \in \mathbb{R}_+^n$ 和 $b \in \mathbb{R}_+^m$ ，EOT 求解

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij} (\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \quad (1)$$

令 $K = \exp(-C/\varepsilon)$ ，Sinkhorn 交替更新对角缩放，从而形成 $\pi = \text{diag}(u)K \text{diag}(v)$ [3]。坐标方法、松弛方法、低秩方法和Newton 方法降低单次求解内部的算术量或迭代次数[1, 8, 13, 14]。融合流式实现把稳定化更新改写为LogSumExp reduction，从而降低显存流量[17]。DRAIN-SINKHORN 保留所选的内部更新，只改变下一次更新实际接收的问题集合。

初始化与重复coupling。Carry、解析延拓和学习得到的dual proposal 可以降低新问题的初始缺陷[2, 15]。当支撑集改变时，大coupling Flow Matching 也使用soft c -transform [20]。这些机制会改变各问题所需的迭代次数。DrainSinkhorn 同时适用于两类情形：正式Packer19 归因实验在所有实验臂中使用相同的固定proposal，而ImageNet-32 训练实验使用相互独立的冷启动coupling。

动态批处理与批量求解器。自动批处理跟踪数据依赖程序中每个成员所在的程序位置[12]。Orca 以一次迭代为单位调度自回归推理[19]。在OT 之外，Ginkgo 的batched CG 提供了相关的执行先例：形状一致的独立线性系统按照各自的残差检验退出，融合solver 的后续迭代会跳过已收敛系统[7]。jaxipm 围绕异质迭代融合和已完成问题替换重新设计IPOPT [16]。在OT 内，POT 的solve_batch 对以批量cost matrix 表示的独立OT 问题进行并行求解；OTT-JAX 则使用JAX transformation 向量化多个Sinkhorn 求解[4, 6, 11]。固定宽度的多问题Sinkhorn 执行是本文的起点。本文处理的是此后如何移除已完成的EOT 问题，同时保留在线成对交互，而不为每个问题实例化cost 张量。

DrainSinkhorn 针对的是EOT 批次的数值结构。交替缩放允许使用当前边缘进行初步检查；双边验证器在问题输出上执行；动态压缩必须保持势函数、边缘、

支撑描述符和scratch state 之间的批元素对应关系。通用masking 不会完成这些EOT 专用状态变换，也不会改变宽度。Table 1 比较已有系统的批处理对象、完成信号和执行动作。我们的OTT-JAX 与PyKeOps 版本移除已完成问题，而packed Triton 实现还使用当前边缘进行初步检查。

OT 消费者。OTT 提供广泛的transport API [4]。Multisample Flow Matching 使用OT coupling 选择训练配对[10]；这些coupling 可以独立于网络优化提前准备[20]。我们在工业端点上评估从OT coupling 到下游消费者的完整路径，并让同一执行路径贯穿固定更新数的特征空间OT-FM 训练流水线。

3 DrainSinkhorn

从固定宽度批次到动态压缩。标准批量执行并行应用 W 个独立Sinkhorn 映射，并一直保持宽度 W ，直到需要最多迭代的问题完成。DrainSinkhorn 以这一固定宽度布局为起点。对问题 t ，令 n_t 为它首次在已配置验证时点通过双边验证器时的批量迭代编号。随后，DrainSinkhorn 写出其结果，并动态压缩所有逐问题状态。因此，批量更新 k 按存活宽度

$$A_k = |\{t : n_t \geq k\}|$$

执行。检查间隔是相邻两次验证器检查之间的批量迭代次数。逻辑masking 在宽度 W 上记录完成情况；压缩后的执行路径则按存活宽度 A_k 运行后续kernel。固定宽度/动态压缩对照测量释放与动态缩宽的净效应。LM/DC 固定释放决策以隔离动态缩宽。实验分别报告宽度与分组干预。

3.1 标准批量Sinkhorn 执行

令 F_t 表示问题 t 的一次完整Sinkhorn 循环，其中包含该问题的边缘、kernel 算子和数值表示。标准batching 增加一个前导问题批次维，但不引入跨问题reduction。在理想算术下，其代数性质为

$$\tilde{F}(\text{pack}(z_1, \dots, z_W)) = \text{pack}(F_1(z_1), \dots, F_W(z_W)). \quad (2)$$

因此，批量kernel 保留EOT 目标和逐实例Sinkhorn 映射，只改变其执行布局。停止检查后，DrainSinkhorn 过滤问题批次维，并对存活问题应用式(2)。

对于一个候选，Sinkhorn 循环执行

$$u^{k+1} = a \odot (K v^k), \quad v^{k+1} = b \odot (K^\top u^{k+1}), \quad (3)$$

其中 $\odot$ 表示逐元素除法。第二个更新之后，理想算术立即给出

$$v^{k+1} \odot (K^\top u^{k+1}) = b. \quad (4)$$

列边缘由构造保证为最新，而行边缘 $u^{k+1} \odot (K v^{k+1})$ 仍可能超出容差。DrainSinkhorn 在一次批量操作中计算所有active 问题的行残差，并且只把预检为正的问题送入双边验证器。

表 1: OT 软件已经提供固定宽度的多问题执行。DrainSinkhorn 的比较点是问题满足完成规则后采取的执行动作。

系统	批量数值对象	完成信号	执行动作	输出检查
POT [6, 11]	使用批量 cost matrix 的独立 OT 问题	求解器停止条件	固定宽度并行求解	求解器残差条件
OTT-JAX vmap [4]	向量化的独立 Sinkhorn 问题	逐问题停止或共享迭代预算	固定宽度向量化执行	求解器误差函数
jaxipm [16]	独立非线性程序	异质迭代融合	替换已完成问题以维持吞吐	非线性程序收敛检查
DRAIN-SINKHORN	具有兼容批量状态的正 EOT 问题	使用当前边缘进行初步检查, 随后执行双边际验证器	从 potential 、 support 、 marginal 与 scratch buffer 中移除通过的问题	当前行、列边缘残差容差

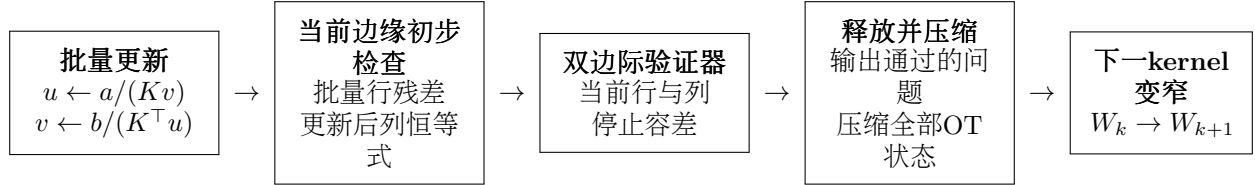


图 1: DrainSinkhorn 的执行循环。使用当前边缘进行初步检查, 可以避免许多完整 plan 计算。通过双边际验证器的问题会离开所有逐问题张量; 下一次批量更新只处理存活问题。

3.2 筛查、验证与压缩

令 $\hat{r}_{t,k}^{\text{row}}$ 表示批量预检值。当 $\hat{r}_{t,k}^{\text{row}} \leq \tau_{\text{screen}}$ 时, 该问题的预检为正; 否则它继续执行, 而不实例化成本更高的 plan-level 检查。对于预检为正的问题, 实现重新计算两侧边缘, 并应用配置的停止规则

$$r_{t,k} = \max\{\|\pi_{t,k} \mathbf{1} - a_t\|_1, \|\pi_{t,k}^\top \mathbf{1} - b_t\|_1\} \leq \tau_{\text{stop}}. \quad (5)$$

被拒绝的问题保持 active, 并在后续验证时点再次检查。false-positive 预检会不必要地调用双边际验证器; false-negative 预检会延迟移除。Section 5 通过重放筛选结果并逐一使用验证器检查来测量这些效应。

对于每个通过的问题, DrainSinkhorn 写出结果, 并压缩所有逐问题对象: source 与 target 描述符、边缘、dual potential、centered shift、log weight 和 scratch buffer。下一次批量更新只看到存活问题。逻辑 mask 对照使用相同的更新、初步检查、双边际验证器、容差和观测完成时间。它冻结通过问题的势函数, 但仍在批次张量中保留该问题, 因此后续 kernel 仍按原始宽度执行。逻辑 masking 与动态压缩的比较由此隔离缩小批宽所移除的工作。

式(2) 与 式(4) 定义理想算术下的变换。固定工作 parity、实际残差、阈值附近压力测试和应用端点共同刻画其实现行为。

保持停止规则的释放。 问题只有通过所选后端使用的同一个双边际验证器后才会被移除。使用当前边缘进行的初步检查只决定是否调用该验证器, 并不授权移除。因此, DrainSinkhorn 保留后端已配置的 release authority, 只改变未完成问题的调度与存储方式。正式计时的 packed Triton 路径在该后端支持的精度下应用这一规则。独立 FP64 replay 作为有限精度决策行为的验证参照。

3.3 执行模式与 fallback

实现提供三种批量执行模式。固定宽度 batching 更新所有问题, 直到共同的最终边界。逻辑 masking 记录逐

问题完成情况并冻结已完成状态, 但不缩窄 kernel。动态压缩移除通过的问题。宽度决定批量 kernel 是否高效, 而批量窗口内各问题所需的迭代次数决定动态压缩可以移除多少宽度。正式实验在计时前固定执行模式、宽度、检查间隔和 fallback。正式实验采用 fail-closed 收敛处理: 达到迭代上限或出现非有限状态时抛出错误。Packed Triton 实现仅在 dynamic-compaction 路径的存活宽度低于预先配置的 `min_packed_width` 时, 才可把窄尾交给匹配的单问题求解器。C66 与 Packer19 正式配置将该阈值设为一。

当一个完整问题能够放入一个加速器时, 不同窗口会送往独立的完整设备 worker, 从而让每个设备拥有独立更新路径, 并避免逐更新 collective。按行划分矩阵仍是单个超大问题的容量路径; 主要 scale-out 路径把完整问题分配给独立设备。

4 存活几何与硬件转化

执行器观测每个问题首次通过验证器的时刻。这些迭代编号定义存活曲线, 并由此确定可移除工作。后端宽度成本曲线进一步决定工作量减少中有多少能够转化为墙钟节省。

4.1 计算被移除的问题更新槽位

对问题 t , 令 n_t 为它首次在已配置验证时点通过双边际验证器时的批量迭代编号, 并令

$$A_k = \{t : n_t \geq k\} \quad (6)$$

表示 packed update k 之前的存活宽度。实测停止检查提供这两个量。

命题1 (问题更新槽位核算). 对于含有 W 个候选的有限窗口,

$$S_{\text{active}} = \sum_{k=1}^{n_{\text{max}}} A_k = \sum_{t=1}^W n_t, \quad S_{\text{static}} = W n_{\text{max}}. \quad (7)$$

因此，相对于具有相同最终边界的固定宽度批次，动态压缩恰好移除

$$R = Wn_{\max} - \sum_{t=1}^W n_t \quad (8)$$

个候选更新槽位。

Proof. 使用 $n_t = \sum_{k \geq 1} \mathbf{1}\{n_t \geq k\}$ ，并交换两个有限求和的顺序。固定宽度执行让全部 W 条 lane 一直保持到 $n_{\max}$ 。□

逻辑masking 会改变哪些状态发生更新，但不会改变kernel 看到的宽度，因此仍保留 $Wn_{\max}$ 个槽位。静态矩形与生存曲线之间的面积，正是动态压缩在运行结束后实际移除的工作。检查间隔已经反映在观测到的 n_t 中；在更密检查时点上首次通过的迭代编号会定义另一个量。

命题2 (任务无关的机会条件). 令 $(n_1, \dots, n_W)$ 为任意有限迭代编号随机向量。那么 $\mathbb{E}[R] > 0$ 当且仅当存在某次批量迭代 k 满足

$$\Pr(0 < A_k < W) > 0. \quad (9)$$

Proof. R 非负，并且仅当每条 lane 具有相同迭代编号时为零。不同的有限迭代编号会产生一个同时含有存活与已完成 lane 的 k 值；反向结论直接由 A_k 的定义得到。□

该条件取决于packed window 内迭代编号的联合分布。即使不同窗口之间所需的迭代编号发生变化，完全同步的 lane 也不会产生可移除工作。

4.2 实测后端上的精确墙钟核算

令 $c_k(a, \xi_k)$ 为匹配更新 k 在宽度 a 时的成本，其中 ξ_k 记录支撑形状、数据类型、kernel 配置以及对照中保持不变的其他后端状态。则

$$T_{\text{active}} = \sum_k c_k(A_k, \xi_k) + H_{\text{screen}} + H_{\text{check}} + H_{\text{compact}} + H_{\text{other}}, \quad (10)$$

H_{other} 包含其余计时内控制工作。令 H_{fixed} 为对应的固定宽度控制成本，并令 ΔH 表示 active 控制成本减去 H_{fixed} 。

命题3 (机会-转化分解). 对于具有相同最终验证边界和匹配更新状态 ξ_k 的固定宽度/动态压缩对照，

$$T_{\text{fixed}} - T_{\text{active}} = \sum_{k=1}^{n_{\max}} [c_k(W, \xi_k) - c_k(A_k, \xi_k)] - \Delta H. \quad (11)$$

因此，动态压缩更快当且仅当

$$\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > \Delta H. \quad (12)$$

Proof. 写出 $T_{\text{fixed}} = \sum_k c_k(W, \xi_k) + H_{\text{fixed}}$ ，减去式(10)，再合并控制项即可。□

这一分解呈现两道门槛。 n_t 之间的差异创造产生正可移除工作所需的存活宽度；实测函数 $c_k(a, \xi_k)$ 决定这些宽度缩减是否跨入更低的硬件成本层级。若所有 k 都满足 $c_k(A_k, \xi_k) = c_k(W, \xi_k)$ ，动态压缩会减少问题更新槽位，却不会节省更新时间；若总成本下降超过 ΔH ，则产生净墙钟收益。

4.3 波次形状的宽度成本

对于本文使用的packed Triton 更新，其launch grid 含有

$$B(a, n) = a \left\lceil \frac{n}{b_m} \right\rceil \quad (13)$$

个thread block，其中 b_m 是行tile 大小。GPU 调度按受occupancy 影响的波次处理该网格[9, 18]。因此，一个有用的局部模型是

$$c(a; n, \xi) \approx g_{n, \xi} \left(\left\lceil \frac{B(a, n)}{Q_\xi} \right\rceil \right), \quad (14)$$

其中 Q_ξ 是该kernel 实测的驻留block 容量， $g_{n, \xi}$ 把wave 数映射为时间。寄存器、共享内存和kernel 状态通过 Q_ξ 进入模型；该数值针对所选设备与kernel 配置进行校准。

在 $b_m = 64$ 的实测A100/Triton 配置上，第一波估计将饱和宽度分别放在 $n = 1024$ 时的约6.75、 $n = 4096$ 时的约1.69，以及 $n = 16384$ 时的一以下。直接原语测量在前两个规模上分别把首次大幅增长定位在宽度 $6 \rightarrow 7$ 与 $1 \rightarrow 2$ ； $n = 16384$ 曲线从宽度一开始就对宽度变化作出响应。这些测量为该后端实例化 $c(a)$ ，并解释为何相同槽位缩减在不同问题规模上具有不同的墙钟价值。

5 实验

所有加速比均定义为基线时间除以DrainSinkhorn 时间。OT 阶段时间直接测量，或通过从包含OT 的已记录区间中减去配对消费者时间来重构。对于重构结果，先在每个配对计时单元内部减去消费者时间，再计算比率、区间或bootstrap 汇总。匹配组件比较固定输入、proposal、容差、验证检查时点、双边验证器和消费者。完整路径比较在两臂中使用相同的packed 或vectorized 后端。当停止控制器产生不同的实际残差时，论文同时报告实际残差和消费者输出偏差。

因此，本文报告的比率具有两个因果作用：

$$\underbrace{\text{固定宽度} \rightarrow \text{动态压缩}}_{\text{释放与动态压缩}} \\ \underbrace{\text{逻辑mask} \rightarrow \text{动态压缩}}_{\text{宽度效应}}$$

固定宽度/动态压缩测量从普通固定宽度批次到动态压缩控制器的净变化；logical-mask/dynamic-compaction (LM/DC) 则在匹配释放决策下隔离动态缩宽。Figure 2 报告实测宽度成本曲线，表 2 按初始宽度组织已登记效应。附录表 10 列出每个对照的实现、计时范围和统计单元。冻结workload 上的bootstrap 区间量化给定冻结输入和执行配置下的计时可复现性。每个对照均在所列实验及测量区间内解释。

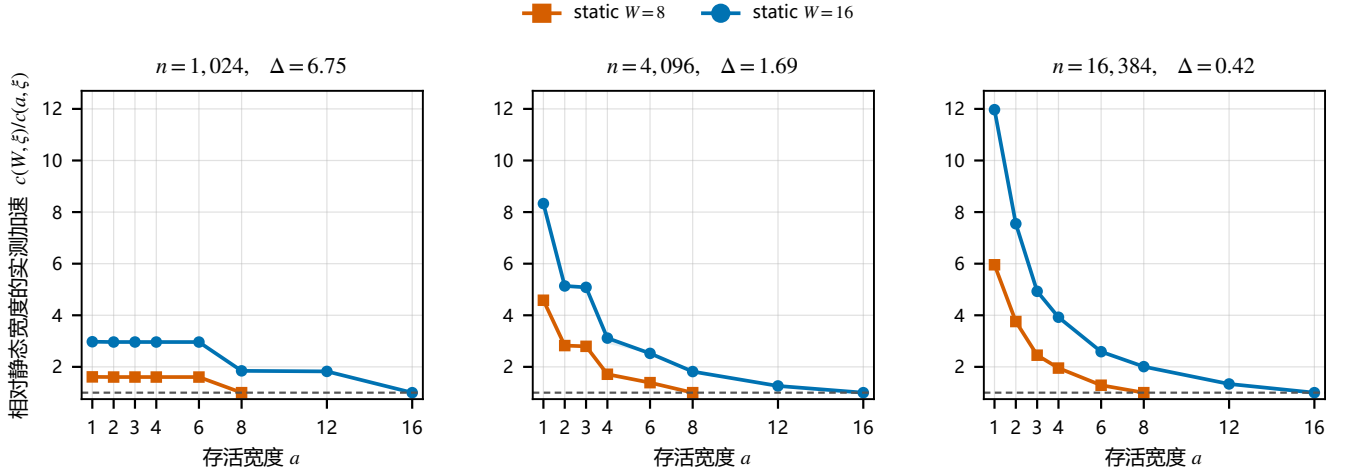


图 2: 从存活宽度到packed-update 加速的实测转化。每条曲线报告相对静态宽度 $W \in \{8, 16\}$ 的 $g_W(a) = c(W, \xi)/c(a, \xi)$; 所有面板使用相同坐标轴, 阴影由30 次重复的逐宽度四分位端点传播得到。面板标题给出决定scheduling-wave 量化的 $\Delta = b_m N_{SM}/n$ 。

表 2: 按初始宽度分组的已登记匹配结果。加速比定义为基线时间除以动态压缩时间。方括号为已登记95% 区间, 圆括号为观测seed 或plan 范围。各行保留所属campaign 的计时范围, 不跨行合并。逐行来源记录在support/figure_materials/MAIN_RESULTS_BY_WIDTH.csv。

W	Campaign	Workload (n)	对照	加速比	计时范围与统计单元
8	C63	ImageNet-32 (1024)	fixed/DC	1.054× (1.033–1.072)	solver field; 3 个训练seed
8	C62	ImageNet-32 (1024)	fixed/DC	1.105× (1.079–1.156)	coupling 准备; 3 个problem-plan seed
8	C80-LM	Packer19 (16384)	LM/DC	1.5397× [1.5389, 1.5404]	扣除consumer 的OT 阶段; 24 个配对单元
8	EXP-P19	Packer19 (16384)	LM/DC	1.5713× [1.5701, 1.5726]	直接OT 阶段; 24 个配对单元
8	C67-F3	MetroPT-3 (16384)	fixed/DC	2.159×	八worker 完整端点; 每worker 固定workload
8	C67-F2	MetroPT-3 (16384)	fixed/DC	3.110×	八worker 完整端点; 每worker 固定workload
16	C33	ImageNet-32 (4096)	fixed/DC	1.421×	稳态关键路径; 40 个窗口
16	C55	ImageNet-32 (4096)	fixed/DC	1.581×	匹配的20-update phase; 10 次重复
16	C67-32k	MetroPT-3 (32768)	fixed/DC	1.616×	单worker 端点; equal-host 几何均值
16	C66	MetroPT-3 (16384)	fixed/DC	1.7447× [1.7434, 1.7467]	双主机ready-arrays-to-consumer 端点
16	C82	MetroPT-3 (16384)	fixed/DC	1.7455× (1.7453–1.7460)	四worker makespan; 3 个已登记plan
16	C67-8k	MetroPT-3 (8192)	fixed/DC	2.316×	单worker 端点; equal-host 几何均值

5.1 匹配的释放与动态压缩

匹配比较从常规固定宽度批量执行开始。C82 在一台主机的四个完整设备worker 上直接比较两条当前MetroPT-3 路径。动态压缩在ready-arrays-to-consumer 端点上达到1.746×, 同时问题更新槽位从5696 降至3128。三个预声明执行plan 均支持动态压缩路径, 描述性范围为1.7453–1.7460×。两臂在每个repeat 中都调用16 个verifier candidate、验证16 个candidate, 并启动32 个plan-application kernel。相应的四GPU 能耗比为1.779×。

C66 提供独立的双主机匹配研究。其固定宽度/动态压缩比在E2 端点上为1.744723×; 在每个host-plan 单元内扣除配对consumer 后, E1 比率为1.754×。六个host-plan 单元均支持动态压缩路径。

C67 把相同的完整设备设计扩展到两台主机上的八个独立worker。两个fixed-per-worker 端点单元的匹配固定宽度/动态压缩加速分别为3.110× 与2.159×, 同时按表 3 降低slots。另有两个在两台主机均通过冻结门的单worker support-size 单元: $n = 8192$ 时为2.316×, $n = 32768$ 时为1.616×。

在C63 中, 每个训练step 使用真实ImageNet-32 PCA500 特征coupling。表 9 报告四条路径的solver 阶段时间。配对solver-field 固定宽度/动态压缩为1.054× (观测seed 范围1.033–1.072×; 表 11)。在完整fixed-update 训练区间上, 固定宽度/动态压缩为1.036× (范围1.030–1.043×)。C62 单独测量三个problem-plan seed 上的coupling 准备, 固定宽度/动态压缩为1.105×。使用一个初始化处理多个问题的实验给出非冷启动条件:

表 3: C67 在两台主机的八个完整设备worker 上执行fixed-per-worker 实验。时间汇总完整consumer 端点。Slots 统计跨worker 的问题更新; 能耗使用同一端点。

路线	固定宽度(s)	动态压缩(s)	Slots 固定/动态	时间比	能耗比
failure-2	12.523	4.026	4736 / 1276	3.110×	3.437×
failure-3	7.999	3.705	3008 / 1244	2.159×	2.292×

在40 个受控重叠support 窗口上, 动态压缩路径加速1.421×、slots 减少30.62%, 并在全部配对窗口中获胜。

这些数值质量结果在NFE 4、8 与16 下始终使用相同的三个配对训练seed。完整逐seed 测量值与分析文件已放在公开GitHub 仓库 (<https://github.com/cis-acorniticacid/DrainSinkhorn>) 中。该评估测量ImageNet-32 PCA500 特征空间OT-FM 端点。

5.2 从宽度到kernel 时间的实测转化

我们通过在固定问题规模下调用同一个packed Triton 更新、只改变宽度, 测量式(11) 中的宽度成本项。Figure 2 将实测中位数归一化为静态宽度 $W = 8$ 与16 下的 $g_W(a) = c(W, \xi)/c(a, \xi)$, 并统一使用存活宽度坐标 $a \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ 。对实际存活宽度 A_k , 式(11) 中对应的单项为 $c(W, \xi_k)[1 - g_W(A_k)^{-1}]$ 。面板值 $\Delta = b_m N_{SM}/n$ 预测平台尺度: $n = 1024$ 时转化受到明显量化 ($\Delta = 6.75$), $n = 4096$ 时量化变细 ($\Delta = 1.69$), $n = 16384$ 时接近连续转化 ($\Delta = 0.42$)。

该诊断隔离packed update, 因此估计的是 $c_k(a, \xi_k)$ 的一个组成部分, 而不是完整执行器。它在 $n = 1024, W = 8$ 下的形状与较小的C63 solver-field 比率一致: 存活宽度进入 $A \leq 6$ 区域后, 继续移除lane 对槽位数的影响大于对packed-update 时间的影响。 $n = 16384$ 的响应则提供较大MetroPT-3 动态压缩收益所需的硬件条件; 匹配的C82 端点进一步测量包含控制与消费者工作的完整净效应。

跨后端家族迁移。 OTT-JAX 与PyKeOps 结果通过独立实现测试执行原理。在真实ImageNet-32、 $W = 16$ 上, 每个固定阶段只处理剩余问题的OTT-JAX 执行比原生OTT-JAX 快2.600×; 匹配的固定宽度/动态压缩对照将其中1.545× 归因于只在阶段边界移除已完成问题并重新划分存活问题。在真实A2D2 LiDAR、 $W = 8$ 上, PyKeOps active 路径比sequential carry 快3.174×。真实ImageNet-32 PyKeOps 单元仍取得1.415× 正收益。tiled-log 实现在不实例化候选kernel 的情况下扩展到 $n = 65536$, 给出实测容量边界。

独立OTT-JAX 与PyKeOps 实现汇总于表 7。

5.3 Packer19 停止与执行Pareto

双主机EXP-PACKER19 实验评估 10^{-4} 、 3×10^{-4} 、 10^{-3} 、 3×10^{-3} 和 10^{-2} 残差容差, 并且不跨容差聚合。所有实验臂使用相同的双边际验证器与残差容差, 以及相同的紧参考消费者输出。

在 10^{-4} 至 3×10^{-3} , 动态压缩相对逻辑masking 提升1.480–1.638×。在 10^{-2} , 全部八个问题都在第8 次更新首次通过, 因此不存在可移除的padding, LM/DC 为中性的0.9995× (95% CI [0.9987, 1.0003])。这些有效对照刻画EXP-PACKER19 的动态压缩路径与逻辑mask 路径; 较早C80-LM 的对照与阶段成本分别见表 5 和 12。

所有已登记端点质量检查均通过。

5.4 匹配组件归因

释放与动态压缩的效应。 C82 在匹配MetroPT-3 E2 端点上得到1.746×; C66 在单独测量的E1 OT 阶段上得到1.754×。在C63 ImageNet-32 四臂训练消融中, 动态压缩在三个seed 上相对固定宽度batching 增加1.036× (区间1.030–1.043×)。这是E3 端点归因; 1.054× 的C63 solver-field 对照单独报告在表 9 和 11。

C80 与C80-LM 在两台主机上使用相同的Packer19 单细胞转移workload, 每台主机包含四张A100-SXM4-80GB。三种冻结方法顺序产生24 个配对的host-order-GPU 单元。每个单元使用排除warmup 后的重复中位数、相同输入、packed-kernel source tree、 10^{-3} 停止容差和转移消费者。

检查与screening。 对照暴露出不同工作。批量执行双边际验证器以及使用当前边缘进行初步检查, 在匹配路径下降低OT 阶段时间。每个C80 比率都先减去其配对消费者时间, 再形成比较。

我们重放全部1488 个预检为负的事件, 并逐一使用双边际验证器检查, 发现其中0 条lane 已经可以释放。初步检查与验证器的行残差最大差异为 1.86×10^{-7} 。每个输出plan 都以 10^{-3} 通过双边际验证器。

逻辑masking 与动态压缩。 在C80-LM 中, 逻辑masking 为中性, 因为packed kernel 仍保持宽度8。动态压缩降低观测宽度, 移除256 个问题更新槽位中的100 个, 并产生1.540× OT 阶段加速, 区间为1.5389–1.5404×。全部24 个配对单元均支持动态压缩。在24 个host-order-GPU 单元中位数上, 端点能耗从879.9 J 降至688.1 J, 而压缩后的buffer 会把峰值allocated memory 从80.8 MiB 提高到106.9 MiB。

5.5 运行区间: 异质性与宽度

M3 固定32 个真实MetroPT 问题的集合, 只改变它们进入窗口的分组。离线逐循环迭代编号 q_t 定义分组, 验证器给出的迭代编号 n_t 测量实际执行工作。Oracle homogeneous grouping 最小化每个窗口内所需迭代编号的差异。Heterogeneous 与random grouping 增大静态矩形和由 n_t 定义的生存曲线之间的面积。匹配的固定宽度/动态压缩比率从1.105× 上升到1.217×, random layout 上为1.235–1.271×。

独立有限精度压力测试。 C10 在1/2/4/8 GPU 上测量84 个阈值附近候选和16 个运行时攻击的决策行为。相对独立FP64 replay, raw FP32 与TF32 各有三个accept decision 不一致; outward-bound guard 随后执行FP64

表 4: 经过50k 次训练更新后的ImageNet-32 PCA500 特征空间质量。表中给出匹配固定宽度 (Fixed) 与动态压缩 (Dynamic) 执行的三个seed 均值; 三项指标均为越低越好。质量评估始终使用相同的三个配对训练seed; 完整逐seed 测量值与分析文件已放在公开GitHub 仓库中。这些是特征空间指标, 不是像素空间FID。

NFE	曲率		Held-out FM MSE		Projected Fréchet	
	Fixed	Dynamic	Fixed	Dynamic	Fixed	Dynamic
4	5.4524	5.4448	1.07854	1.07859	1.2967	1.2886
8	2.8388	2.8356	1.07854	1.07859	1.5286	1.5224
16	1.4546	1.4528	1.07854	1.07859	1.8182	1.8136

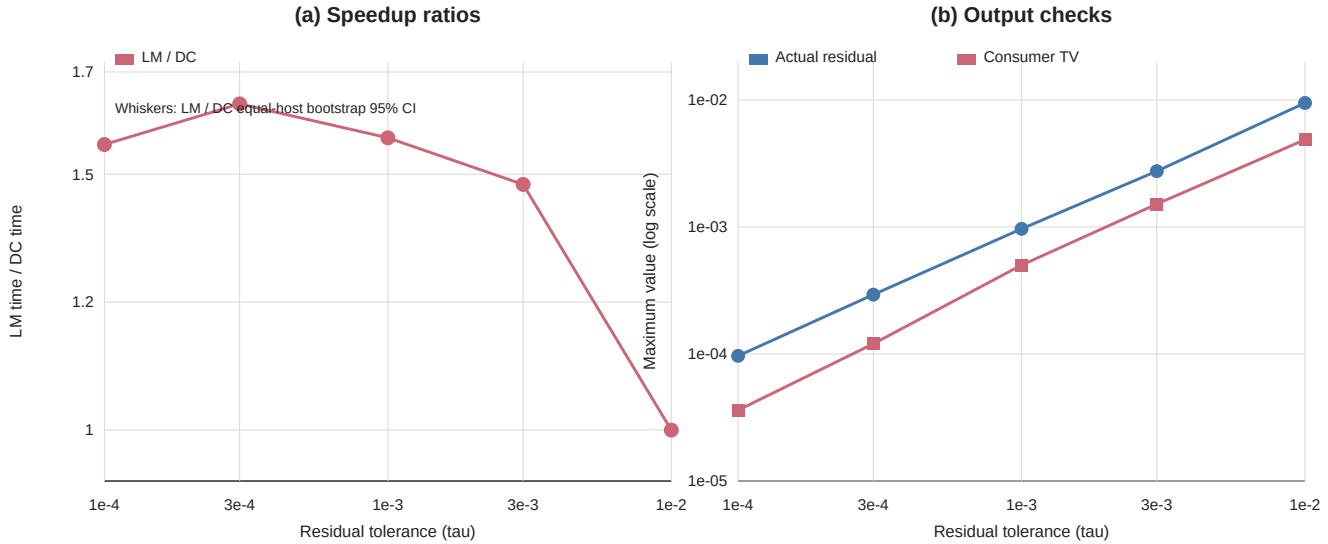


图 3: 使用相同双边际验证器与残差容差的双主机Packer19 停止-执行结果。面板(a) 展示五个容差下的LM/DC 速度比, 误差线为equal-host 分层bootstrap 95% 区间。面板(b) 以对数纵轴展示最大实际残差和最大consumer TV。每个点汇总24 个配对单元; consumer TV 检查相对紧参考的输出差异。

表 5: 在一个冻结输入和release contract 下, Packer19 的单变量归因。每行都报告扣除配对消费者后的OT 阶段时间, 因此各组件效应使用同一测量边界。

改变	匹配对照	范围	比率	证据
批量验证检查	serial / batched verifier	OT 阶段	1.133×	24/24 胜出
当前边缘初步检查	仅验证器 / 先初步检查	OT 阶段	1.105×	24/24 胜出
逻辑冻结	fixed / logical mask	OT 阶段	1.000×	95% CI [0.9998, 1.0003]
动态压缩	logical mask / dynamic compaction	OT 阶段	1.540×	95% CI 1.5389-1.5404×

批量验证检查与当前边缘初步检查两行均为24/24 配对胜出; 这两个对照没有登记独立的E1 bootstrap 区间。

escalation, 在这100 个case 中没有不一致。该guarded 配置与正式计时的应用分开评估。

正式计时的应用与独立worker scale-out。 正式计时的packed Triton 路径使用后端支持的精度和双边际验证器。所有输出plan 均满足记录的行、列边缘残差界限, 且每个端点均通过登记的输出或质量门。C67 把完整窗口分配给独立GPU worker。四个worker 在held-out MetroPT route 上取得3.965-3.979× fixed-per-worker throughput scaling, worker 之间没有Sinkhorn collective; 八worker 的匹配端点分解见表 3。

表 6: 对同一固定真实任务multiset 的M3 分组干预。对于窗口内的离线逐循环迭代编号 q_s , $H_q = \text{sd}(q_s) / \text{mean}(q_s)$, padding 为 $W \max_s q_s / \sum_s q_s - 1$ 。Dynamic slot 是动态压缩相对固定宽度的候选迭代槽位比。表中数值对窗口汇总; timing repeat 用于稳定每种条件, 并非独立数据集。

分组	H_q	Padding	Dynamic slot	固定/动态
heterogeneous	0.236	0.236	0.802	1.217×
homogeneous	0.127	0.110	0.892	1.105×
random 1	0.254	0.293	0.776	1.258×
random 2	0.264	0.270	0.789	1.235×
random 3	0.260	0.279	0.764	1.271×

6 适用区间与组合方式

DRAIN SINKHORN 适用于具有兼容批量表示及双边际验证器的批量正EOT 问题。一个批次中的问题需要不同数量的迭代时会产生可移除工作; 净加速取决于批宽与实测运行时间之间的关系, 以及增量控制与状态移动成本(式 (12))。

初始化同时影响总迭代次数及其在窗口内的差异。使用一个初始化处理多个问题的实验采用受

控重叠的ImageNet-32 support: active 执行使slots 减少30.62%, 包含proposal 与传输成本的steady-state 关键路径加速1.421 $\times$ 。特征训练实验对独立重采样的OT-FM coupling 使用cold proposal。

执行器根据问题是否通过已配置停止规则来进行调度。分组干预利用离线oracle 迭代编号, 测量固定问题集合在不同分组下的执行效应。宽度成本扫描单独测量后端如何把每个存活宽度转换成更新时间。两者共同实例化式(11) 中的两项; 当完成深度估计可用时, 预测分组可以使用相同的量。

动态压缩以状态移动和buffer 换取计算减少。匹配logical-mask/dynamic-compaction 实验报告额外峰值分配26.1 MiB, 同时运行时间与端点能耗下降; 表 12 报告其阶段成本。OTT-JAX 与PyKeOps 通过后端特定的rebucketing 和独立问题批量执行实现释放。

校准后的宽度成本曲线对应指定设备、kernel 路径、问题形状与数值配置。Packed Triton 路径还会将窄尾切换到匹配的单问题求解器, 因此其完整成本曲线包含这一路由边界。核算恒等式与式(11) 适用于任意实测成本曲线; 观测到的A100 wave 位置适用于所报告的Triton 配置。

7 结论

DRAINSINKHORN 从各问题需要不同迭代次数的批次中移除padding OT 工作。该方法以独立Sinkhorn 问题的标准固定宽度执行为起点; 所选求解器的停止规则判断问题是否完成, 动态压缩则使此后的逐问题kernel 变窄。生存曲线恒等式计算消失的问题更新数。机会-转化分解随后将存活宽度映射到实测后端成本曲线并减去控制开销, 从而给出匹配路径的精确break-even 条件。

匹配的四worker MetroPT-3 端点达到1.746 $\times$, ImageNet-32 训练solver 字段达到1.054 $\times$, 已登记的MetroPT scale cell 达到1.616–3.110 $\times$ 。多个问题使用一个初始化的非冷启动条件达到1.421 $\times$, 并减少30.62% 的问题更新槽位。独立的OTT-JAX 和PyKeOps 实现通过各自后端移除已完成问题。

匹配对照识别了加速机制。批量检查与Sinkhorn 专用预检降低检查成本; 逻辑masking 保持kernel 宽度不变; 动态压缩移除OT 工作。在已登记的MetroPT 与Packer19 端点窗口中, 动态压缩同时降低实测GPU 能耗。在保持总迭代次数不变的条件下进行分组, 结果说明窗口内所需迭代编号的差异越大, 可移除padding 越多。直接A100 宽度扫描呈现波次形状的转化曲线: 较小宽度可能共享同一成本层级, 而更大的问题会更早地响应宽度缩减。报告的应用把速度与实际残差以及消费者或训练端点配对呈现。快速内部kernel 让每次更新更便宜; DrainSinkhorn 则不再向已完成问题发出这些更新。

可复现性。 发布制品记录了源代码和输入身份、命令、软件与GPU 环境、原始输出、数值检查和配对分析。

参考文献

- [1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html.
- [2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL <https://proceedings.mlr.press/v202/amos23a.html>.
- [3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL <https://arxiv.org/abs/1306.0895>.
- [4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL <https://arxiv.org/abs/2201.12324>.
- [5] Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun-ichi Amari, Alain Trounev, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL <https://proceedings.mlr.press/v89/feydy19a.html>.
- [6] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boissunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL <https://www.jmlr.org/papers/v22/20-451.html>.
- [7] Ginkgo Project. Batched CG. Ginkgo user guide, 2026. URL <https://gko-project.org/user-guide/concepts/batched/cg.html>. Accessed 2026-09-08.
- [8] Mete Kemerttas, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL <https://arxiv.org/abs/2504.02067>.
- [9] *CUDA C++ Programming Guide*. NVIDIA, 2024. URL <https://docs.nvidia.com/cuda/archive/12.5.0/cuda-c-programming-guide/index.html>.
- [10] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of*

the 40th International Conference on Machine Learning, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL <https://proceedings.mlr.press/v202/pooladian23a.html>.

- [11] POT developers. POT 0.9.6 release notes. Python Optimal Transport documentation, 2025. URL <https://pythonot.github.io/releases.html>.
- [12] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.
- [13] Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL <https://proceedings.mlr.press/v139/scetbon21a.html>.
- [14] Alexis Thibault, L  na  c Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn–Knopp algorithm for regularized optimal transport, 2017. URL <https://arxiv.org/abs/1711.01851>.
- [15] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL <https://proceedings.mlr.press/v206/thornton23a.html>.
- [16] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL <https://arxiv.org/abs/2606.26341>.
- [17] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL <https://arxiv.org/abs/2602.03067>.
- [18] Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 503–521, 2021. URL <https://www.usenix.org/conference/atc21/presentation/yu>.
- [19] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [20] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL <https://arxiv.org/abs/2506.05526>.

A 测量合同与补充细节

A.1 C63 seed provenance

质量评估在所报告的NFE 下始终使用相同的三个配对训练seed: 20260891、20260892 和20260893。完整逐seed 测量值与分析文件已放在公开GitHub 仓库中。

测量层级。 E1 在双边际验证器通过之后, 结束于实测coupling 输出或势函数。E2 包含固定下游消费者。E3 包含训练、生成或任务评估。C82、C66 与C80 记录E2 端点, C63 记录E3 训练端点。正文C82 比率直接使用其登记的E2 端点。C66 与C80 在形成比率和bootstrap 之前, 先从每个单元中减去配对消费者时间; C63 使用记录的solver 字段。EXP-PACKER19 为每个停止控制器和执行模式单元直接记录E1。E2/E3 中不变的消费者或模型时间会稀释对应的OT 阶段收益。C63 的固定宽度/动态压缩内部训练消融比率由完整fixed-update E3 区间计算。

GPU 能耗区间。 GPU 能耗取A100 NVML 累计总能耗counter 在已登记方法调用之前与最终CUDA synchronization 之后的差值。Counter 以毫焦耳报告, 分析将差值换算为焦耳。该ready-arrays-to-consumer 区间包含solver/proposal 与consumer 工作, 并排除编译、warm-up 和prepared-input transfer 组件。不支持该counter 时, 记录为unavailable, 不使用功率上限估计替代。

C82 匹配固定宽度/动态压缩对照。 C82 在相同的MetroPT-3 failure-3 输入、冷启动proposal、正则化、容差、验证检查时点、双边际验证器、packed backend 和固定消费者上运行固定宽度路径与动态压缩路径。实验变量是通过验证器的lane 是否从后续kernel 中动态移除。一台主机提供四个彼此独立的完整设备A100 worker, 不使用worker 间collective 或问题切分。对于每个plan 和计时重复, 端点取四个worker 的ready-arrays-to-consumer 最大时间。每个plan 对两次计时重复取中位数, 报告的 $1.745535 \times$ 是三个plan 比率的描述性几何均值。plan 标识只冻结执行顺序, 不是独立MetroPT 样本。48 个worker-level repeat 全部收敛; 最大行、列 L_1 残差分别为 9.476650×10^{-4} 和 1.283915×10^{-6} 。在冻结消费者下, 两臂的failure-window ROC AUC 都为1.0, 求解器不使用标签。最大配对transport-cost 与barycentric-shift 差分别为0.02792263 和0.00346756。

C66 双主机匹配对照。 在C66 的匹配对照中, 固定宽度路径与动态压缩路径使用相同输入、冷启动proposal、正则化、容差、验证检查时点、双边际验证器、packed backend 和固定消费者; 根据停止规则释放问题与动态压缩是受控差异。三个顺序平衡的plan 会在两台主机各自的四个GPU placement 上复用。GPU placement 与timing repeat 用于稳定端到端测量; 登记的端点比率包含不变的非OT 消费者时间。正文C66 OT 阶段比率在每个host-plan 单元内部减去配对消费者时间。host-plan 组合是该冻结workload 的配对计时单元; placement 和repeat 是timing replica, 而非独

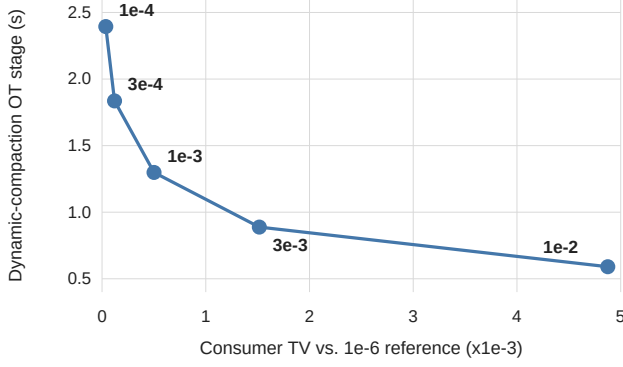


图 4: Packer19 随容差变化的OT 速度-输出前沿, 使用dynamic-compaction 路径。每个点是在该残差容差下24 个配对单元的E1 OT 阶段时间中位数; consumer TV 相对本实验的 10^{-6} 紧参考测量。

立MetroPT workload。对六个有效host-plan 单元的重分析报告分层bootstrap 区间。

EXP-PACKER19 来源。 停止-执行协议的SHA-256 为c010bcf9b3e276c8269514324cc8ed4bc2fc5958c362c7b880e3946dee75075b, 并绑定source commit 96de52b8ceff8fa3c0468bf45567e6b017483e73。两台主机、每台四张A100-SXM4-80GB GPU、三种冻结方法顺序和三次timing repeat 产生768 个正式单元。每个容差具有24 个配对的host-order-GPU 单元和equal-host bootstrap 区间。Consumer TV 用于数值检查输出差异。

执行模式协议。 正式实验在计时前固定representation、width、execution mode、检查间隔、stopping tolerance、checking path、窄尾handoff 阈值与fail-closed 策略。宽度与分组干预映射选择固定宽度执行或动态压缩的条件。

宽度成本诊断来源。 原语扫描使用一张具有108 个SM 的NVIDIA A100-SXM4-80GB, driver 为535.129.03, PyTorch 2.5.1 配CUDA 12.4, Triton 3.1.0, source commit 为73aec2941bfa1a5caa6a6bddf7aa83b734f9bb8b。每次实测shifted_step 调用包含两个packed Triton log-sum-exp kernel, 并排除边缘screen、双边验证器、动态压缩与solve.batch 外层循环。Warm-up 后, width 在30 次repeat block 内随机化。 $n = 1024$ 扫描width 1-8、10、12 与16; $n = 4096$ 和 $n = 16384$ 扫描1、2、3、4、6、8、12 与16。该诊断提供式(11) 中实测宽度成本曲线的packed-update 组成部分。

独立后端实现。 OTT-JAX 与PyKeOps 路径通过各自原生后端机制移除已完成问题。OTT-JAX 执行固定的20-update phase, 在phase boundary 检查, 移除通过lane, 并对存活问题重新分桶; 原生vmap 和匹配的fixed-phase static 路径作为对照。PyKeOps 路径使用一个独立问题批量matrix-free operator、固定几何验证

检查时点、双边验证器, 以及对通过问题状态的动态移除。Sequential 与static-padded PyKeOps 路径使用相同symbolic backend。共享原理是逐问题释放并动态移除已完成项; 检查和packing 机制仍由后端决定。Table 7 中所有跨后端结果都报告实际达到的两侧边缘残差。

A.2 独立后端实现

B 部署与运行边界

完整设备执行与按行划分矩阵。 当一个问题能放入一个加速器时, 完整设备worker 处理不同窗口, 不需要Sinkhorn collective。按行划分矩阵为单个超大问题提供容量路径。完整设备执行是C67 使用的throughput 路径。在一台主机上, 从一个worker 增加到四个worker, 会在两条held-out MetroPT route 上产生 $3.965\text{--}3.979 \times$ fixed-per-worker throughput scaling。双主机八worker fixed-per-worker 实验报告该worker 数下的端点比较。本文把one-to-four 与双主机八worker 作为两个独立scaling study 报告。峰值内存保持为每个worker 的本地内存。

B.1 补充归因与证据边界

C62 固定packed Triton adapter、输入和数值合同, 只改变动态压缩; 三个固定宽度/动态压缩比值为1.156、1.082 和1.079。固定宽度/动态压缩slots 分别为64/52、64/56 和64/56。这些测量覆盖coupling 准备。C55 在每个容差上使用一个冻结ImageNet-32 输入和十次计时重复; phase size 在独立warmup bank 上选择。 10^{-2} 时的 $0.995 \times$ 与匹配phase 的正结果一起保留。

C67 的全局八worker 结果在每个worker 工作量固定的条件下汇总完整consumer 端点。路线2 的固定宽度/动态压缩秒数为12.523/4.026, 路线3 为7.999/3.705; 实际执行slots 分别为4736/1276 和3008/1244。报告的support-size 加速包括路线3 上 $n = 8192$ 与32768 两个在两台主机均通过冻结门的单元。连续transport-cost 与barycentric-shift 差异是在已登记二元consumer 端点下报告的诊断量。

C33 的非冷启动结果在真实ImageNet 特征上使用构造的90% overlap target stream。其canonical 反序warm replay 排除compile、cache load 和进程启动, 同时保留descriptor、传输、anchor、proposal、correction 和verification 成本。M3 的分组比值描述一个固定的32 问题集合; oracle 迭代编号 q_t 定义分组干预。固定集合使 $\sum_t q_t$ 不变, 分组则改变跨窗口的 $\sum_w W_{q_{\max}, w}$ 。这些比率报告由此产生的相对运行时间效应; 每种分组下的绝对实验臂时间可进一步分离固定宽度与压缩路径的变化。

表 7: OTT-JAX 与PyKeOps 在真实数据上的独立实现。每个比率均为基线时间除以压缩路径时间；不确定性标签分别说明timing-repeat range 或observed-input range。

后端	Workload	OT 阶段对照	加速比 (不确定性)	计时：单元	精度合同
OTT-JAX	ImageNet-32, $n = 4096, W = 16$	native vmap / 每个固定阶段处理剩余问题	$2.600\times$ (repeat range 2.338–2.664 $\times$)	direct E1; 6 timing repeats	max L1 9.97×10^{-4}
PyKeOps	A2D2 LiDAR, $n = 8192, W = 8$	sequential carry / active	$3.174\times$ (clip range 2.180–3.944 $\times$)	direct E1; 3 real clips	max L1 9.97×10^{-4}
PyKeOps	ImageNet-32, $n = 16384, W = 4$	sequential / active	$1.415\times$ (seed range 1.360–1.448 $\times$)	direct E1; 3 seeds	max L1 8.93×10^{-4}

表 8: 主要正式workload 配置。各行均使用冷启动，以及检查当前两侧边缘残差的验证器。

实验	端点、数值配置与统计单元
C82	MetroPT-3 预测性维护(E2); $(n, W, \epsilon) = (16,384, 16, .1)$; 检查间隔4; $\tau = 10^{-3}$; 一台主机、四个完整设备worker、三个顺序plan、两次计时重复。
C66	MetroPT-3 预测性维护(E2); $(n, W, \epsilon) = (16,384, 16, .1)$; 检查间隔4; $\tau = 10^{-3}$; 两台主机、三个plan。
C80/LM	Packer19 scrNA 转移(E2); $(16,384, 8, .1)$; 检查间隔4; $\tau = 10^{-3}$; 24 个配对单元。
EXP-P19	Packer19 同控制器Pareto (E1); $(16,384, 8, .1)$; 检查间隔4; $\tau = 10^{-4}$ – 10^{-2} ; 每个容差24 个配对单元。
C63	ImageNet-32 PCA500 OT-FM (E3); $(1,024, 8, .03)$; 检查间隔4; $\tau = 10^{-3}$; 三个配对seed。

表 9: 三个packed-Triton workload 在 10^{-3} 工作点的绝对OT 阶段时间（秒）。Packer19 报告24 个配对host-order-GPU 单元的中位数；ImageNet-32 报告三个配对训练seed 上solver 阶段的均值 $\pm$ 样本标准差；MetroPT-3 报告三个已登记worker-1 plan 单元的中位数。Packer19 不报告固定宽度值，因为其已登记固定宽度实验臂实际执行了compaction 分支，故予以排除。破折号表示没有可用的匹配实验臂。配对效应另行报告。

Workload	固定宽度批次	逻辑mask	动态压缩
Packer19 (EXP-P19; $n = 16384, W = 8$)	—	2.038	1.299
MetroPT-3 (C66; $n = 16384, W = 16$)	14.839	—	8.462
ImageNet-32 PCA500 (C63)	122.17 ± 2.91	—	115.89 ± 0.83

表 10: 所报告归因对照的证据索引。比率均为分子路径时间/分母路径时间；大于1 表示分母更快。GM 为几何均值。各实验的计时范围及样本/重复单元逐行列出。C33 是较早项目adapter 的受控重叠support 实验。C62/C63 的正则化参数为0.03，其余各行均为0.1。完整实验目录及排除记录见来源总账。

实验	后端/ workload	$n, W; \tau$	Proposal	对照	比率	计时范围	样本/ 重复
C82	packed Triton / MetroPT-3	$16384, 16; 10^{-3}$	冷启动	fixed/ dynamic	1.745535	E2 四worker makespan; slots 5696/3128 = $1.820972 \times$	plans 1.745273 / 1.745356 / 1.745975; 2 次重复 host-plan 单元
C66	packed Triton / MetroPT-3	$16384, 16; 10^{-3}$	冷启动	fixed/ dynamic	1.754	OT 阶段; 逐对扣除consumer	3 个训练seed
C63	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	冷启动	fixed/ dynamic	1.054	solver 字段; 补充GM	3 个训练seed
C63	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	冷启动	fixed/ dynamic	1.036	完整固定步数训练	3 个训练seed
C62	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	冷启动	fixed/ dynamic	1.105	coupling 准备	3 个problem-plan seed
C67	packed Triton / MetroPT routes 2,3	$16384, 8; 10^{-3}$	冷启动	fixed/ dynamic	3.110 / 2.159	8-worker 端点; 每worker 工作量固定	2 条路线; host/plan 重复
C67	packed Triton / MetroPT route 3	$8192, 16; 10^{-3}$	冷启动	fixed/ dynamic	2.316	单worker 端点; 等主机GM	2 台主机; 各3 个plan
C67	packed Triton / MetroPT route 3	$32768, 16; 10^{-3}$	冷启动	fixed/ dynamic	1.616	单worker 端点; 等主机GM	2 台主机; 各3 个plan
C55	OTT-JAX / ImageNet-32	$4096, 16; 10^{-3}$	固定	fixed/ dynamic	1.581	匹配20-update 阶段	一个输入; 10 次重复
C55	OTT-JAX / ImageNet-32	$4096, 16; 3 \times 10^{-3}$	固定	fixed/ dynamic	1.556	匹配10-update 阶段	一个输入; 10 次重复
C55	OTT-JAX / ImageNet-32	$4096, 16; 5 \times 10^{-3}$	固定	fixed/ dynamic	1.366	匹配10-update 阶段	一个输入; 10 次重复
C55	OTT-JAX / ImageNet-32	$4096, 16; 10^{-2}$	固定	fixed/ dynamic	0.995	匹配10-update 阶段	一个输入; 10 次重复
C33	packed Triton / ImageNet-32	$4096, 16; 10^{-3}$	共享anchor	fixed/ dynamic	1.421	steady-state 关键路径	40 个受控窗口
C80-LM	packed Triton / Packer19	$16384, 8; 10^{-3}$	冷启动	fixed/LM	1.000	OT 阶段; 逐对扣除consumer	24 个host-order-GPU 单元
C80-LM	packed Triton / Packer19	$16384, 8; 10^{-3}$	冷启动	LM/DC	1.540	OT 阶段; 逐对扣除consumer	24 个host-order-GPU 单元
EXP-P19	packed Triton / Packer19	$16384, 8; 10^{-4} - 3 \times 10^{-3}$	冷启动	LM/DC	1.480–1.638	直接OT 阶段; 四个容差	每容差24 个配对单元
EXP-P19	packed Triton / Packer19	$16384, 8; 10^{-2}$	冷启动	LM/DC	0.9995	直接OT 阶段; 同步释放	每容差24 个配对单元

表 11: 逐seed 的C63 solver 时间（秒）及固定宽度/动态压缩。对已有记录进行描述性配对重分析；三个比值的几何均值为1.054。

Seed	固定宽度(s)	动态压缩(s)	固定/动态
20260891	123.429	116.699	1.058
20260892	124.233	115.934	1.072
20260893	118.840	115.036	1.033

表 12: C80-LM 阶段时间（秒）：匹配实现中各组件的边际中位数。各阶段分别汇总；总时间比率由配对的完整路径计时计算，见表 5。

阶段	LM (s)	PC (s)
修正	1.7063	1.0708
验证检查	0.6385	0.4006
Verifier	0.1045	0.1045
Mask / compaction	0.000306	0.003370